INFLUENCEAI

Complete Product + Software Requirements + Step-by-Step Build Guide

EDXSO AI Engineer Intern – Assignment 1

Automated Micro-Influencer Outreach System

A practical implementation blueprint for building a professional, demo-ready application using Antigravity, React, FastAPI, Supabase and Gemini.

Version 1.0 | Prepared: 10 November 2026

Important: Use real, verifiable public data. Never invent influencer profiles, follower counts, engagement rates, or contact emails. Respect platform terms, robots/access restrictions, privacy rules and API quotas.

1. Project overview

Product name: InfluenceAI – Automated Influencer Outreach Platform

One-line pitch: Discover relevant micro-influencers, qualify and enrich their public profiles, generate personalized collaboration messages, and track outreach from one dashboard.

The application should demonstrate API integration, data extraction, data processing, filtering, LLM-based personalization, workflow automation, persistence, and a polished user interface.

Core workflow

- Create a campaign and define niche, platform, target audience, follower range and campaign objective.
- Discover public influencer/channel profiles from a permitted source (start with YouTube Data API v3).
- Normalize profiles and collect available public metrics with source URLs and retrieval timestamps.
- Calculate engagement where the required underlying metrics are available; otherwise mark it Unknown.
- Classify each profile as Pass, Fail or Review and store the reasons.
- Find a publicly listed business contact email where available. Never guess or generate an email address.
- Use Gemini to draft a tailored email (60–90 words) and Instagram DM (15–30 words).
- Review drafts, simulate sending, and track status, timestamps, errors and duplicate prevention.
- Export the influencer dataset and outreach tracker as CSV.

Recommended first release

Build a reliable YouTube-first workflow before attempting multiple platforms. YouTube provides structured channel and video statistics through its API. It does not guarantee public email availability. For Instagram/TikTok, use only official/approved APIs or manually supplied public profile URLs and permitted public information; do not bypass login, anti-bot controls or access restrictions.

Success criteria

- At least 50 real influencer records if the available API/source and quota allow it; clearly report the actual number collected.
- Every record has a profile/source URL and retrieval timestamp.
- Unknown metrics and unavailable contact details remain explicitly Unknown / Not Found.
- Qualification decisions include transparent reasons.
- Generated messages meet the required word limits and reference profile-specific details.
- Repeated discovery/import does not create duplicate campaign records.
- Demo mode records a simulated send without claiming a real email or DM was sent.

2. Product requirements document (PRD)

Problem and target users

Marketing teams and small brands spend time finding creators, checking relevance, researching profile details, writing personalized pitches and maintaining spreadsheets. InfluenceAI centralizes these steps for a small campaign.

Primary users: campaign manager, influencer marketing associate, founder/brand operator, and reviewer.

Product goals

- Reduce manual discovery and qualification effort.

- Make influencer selection explainable through visible criteria and evidence.
- Generate useful, editable outreach drafts rather than generic mass messages.
- Provide a traceable campaign tracker and exportable results.

In scope

- Campaign creation and configuration
- YouTube creator discovery
- Profile enrichment and metric calculation
- Rule-based qualification
- Gemini message generation
- Draft review and simulated sending
- Outreach status tracking
- CSV export
- Audit/source metadata

Out of scope for MVP

- Scraping behind authentication or bypassing platform restrictions
- Automated Instagram DM sending without approved API access
- Guaranteed email discovery
- Payment/subscription system
- Complex multi-user role management
- Claims of verified audience demographics when no reliable source exists

User stories

- As a campaign manager, I can create a campaign with a niche, follower range and target criteria.
- As a user, I can discover creator profiles and see where each data point came from.
- As a user, I can see why a creator passed, failed or needs review.
- As a user, I can generate and edit a personalized email and DM.
- As a user, I can mark a message as sent, simulated, replied, or not interested.
- As a user, I can export the results and avoid duplicate outreach.

Acceptance criteria

- Campaign configuration is saved and can be reopened.
- Discovery handles API errors, empty results and quota/rate-limit responses gracefully.
- Profile records have stable platform identifiers and canonical profile URLs.
- Engagement is computed only when views, likes and comments are available.
- All qualification rules are visible in the UI and persisted with each decision.
- Gemini output is validated for word count and required personalization fields.
- Duplicate detection is enforced by campaign + platform + platform profile ID.
- CSV export includes profile, qualification, source, contact and outreach fields.

3. Software requirements document (SRD)

Architecture

```
React + Vite + Tailwind (frontend) | REST / JSON FastAPI (backend) |-- Discovery service: YouTube Data API |-- Enrichment service: permitted public business-contact lookup |-- Qualification service: deterministic rules |-- Personalization service: Gemini API |-- Outreach service: simulation / approved email provider | Supabase PostgreSQL (database + optional Auth)
```

Keep API keys on the backend only. The browser must never receive Gemini, YouTube or email-provider secrets.

Suggested technology

- Frontend: React, Vite, Tailwind CSS, React Router, Axios, Recharts or a lightweight chart library.
- Backend: Python 3.11+, FastAPI, Pydantic, SQLAlchemy or Supabase Python client, httpx, python-dotenv.
- Database: Supabase PostgreSQL.
- AI: Gemini API for personalization only; deterministic code handles qualification and validation.
- Exports: Python csv module or pandas.
- Deployment: Vercel/Netlify for frontend; Render/Railway/Fly.io for backend; Supabase hosted database.

Database entities

Table	Important fields
campaigns	id, name, niche, platform, min_followers, max_followers, min_engagement, objective, created_at
influencers	id, platform, platform_user_id, name, profile_url, followers, avg_views, engagement_rate, niche, bio, contact_email, email_status, source_url, retrieved_at
campaign_influencers	id, campaign_id, influencer_id, qualification_status, qualification_reasons, added_at
outreach_messages	id, campaign_influencer_id, email_subject, email_body, dm_body, generation_model, created_at
outreach_events	id, outreach_message_id, channel, status, event_time, error_message, simulated

Qualification logic

Use campaign-configured thresholds. Suggested default: follower count between 5,000 and 100,000 inclusive. Engagement threshold should be configurable rather than hard-coded as a universal benchmark.

```
if followers is None: status = 'Review' elif followers < min_followers or followers > max_followers: status = 'Fail' elif engagement_rate is None: status = 'Review' elif engagement_rate >= campaign.min_engagement: status = 'Pass' else: status = 'Fail'
```

Engagement example: for each eligible recent video calculate ((likes + comments) / views) × 100, then average those video-level rates. Define the sample window (for example, latest 10 eligible videos). Exclude videos with missing/zero views from the calculation and report the sample count. Do not present this as a universal industry metric.

API endpoints

Method + route	Purpose
POST /api/campaigns	Create campaign
GET /api/campaigns	List campaigns
GET /api/campaigns/{id}	Campaign details and summary
POST /api/campaigns/{id}/discover	Discover creators
POST /api/campaigns/{id}/qualify	Run qualification rules
POST /api/influencers/{id}/enrich	Enrich a profile

Method + route	Purpose
POST /api/campaign-influencers/{id}/generate	Generate email and DM
POST /api/outreach/{id}/simulate	Record simulated outreach
PATCH /api/outreach/{id}/status	Update outreach status
GET /api/campaigns/{id}/export.csv	Export campaign tracker

Message generation contract

Ask Gemini to return structured JSON with subject, email_body, dm_body, personalization_facts and safety_notes. Include only verified facts from the influencer record. The email must be 60–90 words and the DM 15–30 words. Validate word counts in Python; if outside range, ask for one correction or flag for manual editing. Never ask the model to invent an email, statistics, audience demographics or brand relationship.

```
SYSTEM: You write concise, respectful creator partnership outreach. Use only the supplied verified
profile facts. Never invent metrics, contact details, audience demographics, past brand work or personal
details. Return valid JSON only. Email body: 60-90 words. Instagram DM: 15-30 words. Mention one
relevant content theme and a clear collaboration idea. Avoid exaggerated praise, pressure, or false
familiarity.
```

Outreach states

Draft → Approved → Simulated Sent / Sent (only if actually sent through an authorized provider) → Replied / No Response / Interested / Not Interested / Bounced. Store channel, timestamp, actor or simulation flag, and provider error when applicable.

4. User interface and dashboard

Use a clean professional SaaS layout: dark navy sidebar, white/light-gray workspace, restrained blue accent, clear typography, generous spacing, responsive tables and visible loading/error states.

Screens

- Dashboard: campaign count, discovered creators, qualified creators, drafts generated, outreach status chart, recent activity.
- Campaigns: campaign list, create campaign modal/page, status and date.
- Create Campaign: campaign name, niche, platform, follower range, engagement threshold, target location (optional), objective and brand description.
- Discovery: search controls, progress state, discovered creator table, source links and import action.
- Influencer Detail: profile summary, metrics, content themes, contact email status, source evidence and qualification reasons.
- Outreach Studio: editable email subject/body and DM, word counters, regenerate button, approve and simulate-send actions.
- Outreach Tracker: creator, channel, status, last activity, next action and filters.
- Export/Settings: CSV export, API connection health and demo-mode indicator.

Visual quality checklist

- No empty-looking dashboard: use honest empty states with a clear first action.
- Use realistic but clearly labelled demo data only for UI preview; never mix it into submitted research dataset.
- Show skeletons/loading indicators during API calls.
- Show actionable error messages and retry controls.
- Tables need search, filters, sort, pagination and responsive overflow.

- Add tooltips for engagement calculation and qualification criteria.
- Make all primary buttons functional; remove decorative controls that do nothing.

5. Exact project structure

```
InfluenceAI/ ■■■ frontend/ ■ ■■■ src/ ■ ■ ■■■ components/ ■ ■ ■■■ pages/ ■ ■ ■■■ layouts/ ■ ■ ■■■ services/ ■ ■ ■■■ hooks/ ■ ■ ■■■ utils/ ■ ■ ■■■ App.jsx ■ ■ ■■■ main.jsx ■ ■■■ package.json ■ ■■■ .env.example ■■■ backend/ ■ ■■■ app/ ■ ■ ■■■ api/routes/ ■ ■ ■■■ services/ ■ ■ ■■■ models/ ■ ■ ■■■ schemas/ ■ ■ ■■■ database/ ■ ■ ■■■ core/config.py ■ ■ ■■■ main.py ■ ■■■ requirements.txt ■ ■■■ .env.example ■■■ database/schema.sql ■■■ data/exports/ ■■■ screenshots/ ■■■ docs/InfluenceAI_Complete_Build_Guide.pdf ■■■ .gitignore ■■■ .env.example ■■■ README.md
```

Environment variables

```
# backend/.env (never commit this file) YOUTUBE_API_KEY=your_key GEMINI_API_KEY=your_key SUPABASE_URL=your_project_url SUPABASE_SERVICE_ROLE_KEY=your_service_role_key DATABASE_URL=your_postgres_connection_string APP_ENV=development OUTREACH_MODE=simulation # frontend/.env VITE_API_BASE_URL=http://localhost:8000
```

Commit only .env.example with placeholder values. Add .env, node_modules, .venv, __pycache__, exports containing private data and build outputs to .gitignore. Never expose the Supabase service-role key in frontend code.

6. Step-by-step build plan in Antigravity

Create a folder named **InfluenceAI**, open it in Antigravity, and build in small phases. After each phase, run the app and fix errors before moving on.

Phase 1 – Scaffold and design

```
You are a senior full-stack engineer. Build the frontend foundation for InfluenceAI. Use React + Vite + Tailwind. Create a polished responsive SaaS dashboard with sidebar navigation and routes: Dashboard, Campaigns, Discovery, Influencers, Outreach Studio, Tracker, Settings. Use reusable components, clean spacing, accessible forms, loading/empty/error states. Use clearly labelled mock data only for visual preview. Do not build fake API success. Provide exact setup and run commands, then verify the app builds.
```

Phase 2 – Backend foundation

```
Create a FastAPI backend under /backend with modular routers, Pydantic schemas, configuration via environment variables, CORS for the local frontend, health endpoint, structured error responses and API docs. Add requirements.txt and .env.example. Do not hardcode secrets. Provide commands to create a virtual environment, install dependencies and run uvicorn.
```

Phase 3 – Database

```
Implement the PostgreSQL schema for campaigns, influencers, campaign_influencers, outreach_messages and outreach_events. Add foreign keys, timestamps, indexes and unique constraints. Implement database connection and CRUD services. Enforce unique campaign + platform + platform_user_id membership to prevent duplicates. Provide schema.sql and migration/setup instructions.
```

Phase 4 – Discovery and enrichment

```
Implement YouTube Data API v3 discovery using the backend only. Search channels by campaign niche/query, retrieve channel details and recent public video stats. Persist source URLs, retrieval timestamps and the API fields used. Calculate engagement only when sufficient metrics exist. Do not fabricate missing data. For contact email, inspect only permitted public business/contact sources; if unavailable store Not Found. Handle quota, rate limits, missing fields and retries.
```

Phase 5 – Qualification and AI drafts

```
Implement configurable qualification rules with Pass/Fail/Review and explicit reasons. Implement Gemini personalization service using only verified profile facts. Return JSON email subject/body and Instagram DM. Enforce 60-90 email words and 15-30 DM words in backend validation. Save prompt/model metadata and drafts. Allow user editing and regeneration. Never invent contact details or metrics.
```

Phase 6 – Outreach simulation and tracker

```
Implement demo-safe simulated sending. A simulation must create an outreach event with simulated=true
and status=Simulated Sent; it must not send a real message. Add status updates, event history, duplicate
protection, filters and CSV export. Only add real email sending if an authorized provider and
credentials are configured; clearly separate it from simulation. Do not automate Instagram DMs without
approved API access.
```

Phase 7 – Connect frontend and test

```
Connect all screens to real FastAPI endpoints. Remove mock data from operational flows. Add loading,
empty, success and error states. Test campaign creation, discovery, qualification, enrichment, draft
generation, editing, simulation, status update, duplicate prevention and CSV export. Fix all
console/backend errors. Ensure no secrets are present in frontend bundle or Git history.
```

Phase 8 – Polish and submit

```
Prepare README with problem, features, architecture diagram, stack, setup, environment variables, API
usage, database setup, limitations, ethics/data sources, screenshots and demo steps. Add screenshots
and a short demo video. Provide a sample exported dataset containing only real verified records. Clearly
label any UI-only sample data as demo data. Run a final clean install and build.
```

7. Local setup commands

Frontend

```
cd InfluenceAI npm create vite@latest frontend -- --template react cd frontend npm install npm install
axios react-router-dom lucide-react recharts npm install -D tailwindcss @tailwindcss/vite npm run dev
```

If Antigravity has already generated the frontend, do not recreate it; use the package.json scripts it created.

Backend

```
cd InfluenceAI python -m venv .venv # Windows PowerShell: .venv\Scripts\Activate.ps1 # macOS/Linux:
source .venv/bin/activate pip install fastapi uvicorn[standard] pydantic-settings python-dotenv httpx
google-api-python-client supabase sqlalchemy psycopg[binary] uvicorn app.main:app --reload --port 8000
```

The exact import path may differ depending on the generated folder structure. Confirm that /health and /docs open successfully.

Run both services

- Frontend: http://localhost:5173 (or the port Vite prints).
- Backend: http://localhost:8000
- API documentation: http://localhost:8000/docs

8. Testing and evaluation checklist

- Create campaign with valid and invalid values; verify validation.
- Discover creators with an API key; test empty search and quota error paths.
- Confirm every real creator has a stable platform ID, source URL and retrieval timestamp.
- Test missing follower/video metrics; verify they become Unknown/Review, not zero.
- Manually verify engagement calculations on several profiles.
- Test Pass, Fail and Review boundaries, including exactly min/max follower counts.
- Generate several drafts; count words and check personalization against source facts.
- Verify missing email never triggers a guessed email or attempted send.
- Simulate sending and verify event history, timestamp and simulated=true.
- Run discovery twice and confirm no duplicate campaign entries.
- Export CSV and inspect headers, rows and encoding.

- Test responsive layout, keyboard navigation, loading states and API failures.
- Search repository history and frontend build for leaked API keys.

Suggested demonstration sequence

1) Open dashboard → 2) create a Fitness & Wellness campaign → 3) discover creators → 4) open a profile and inspect sources → 5) run qualification → 6) generate personalized email/DM → 7) edit and approve → 8) simulate send → 9) inspect tracker → 10) export CSV.

9. Submission package

- GitHub repository with clean commit history and setup instructions.
- Live frontend/backend demo where feasible, or a recorded local demo.
- At least 50 real records if achievable; explain source and any API limitations.
- CSV dataset with profile fields, metrics, source URLs, retrieval timestamps and qualification reasons.
- Outreach message file or database export with generated email and DM drafts.
- Outreach tracker export showing simulated vs actual sends accurately.
- README, architecture diagram, screenshots and known limitations.

Suggested repository name: **influenceai-automated-outreach**.

10. Risks, ethics and practical notes

- API quotas: YouTube Data API quota may limit discovery. Cache responses and show quota errors; do not imply exhaustive coverage.
- Email availability: many creators do not publish contact emails. Report the true missing count; do not guess.
- Platform restrictions: public visibility does not automatically mean unrestricted scraping or automated messaging is permitted. Follow terms and applicable law.
- LLM hallucinations: pass a small verified fact set, require structured output, validate word count and keep human approval.
- Metric comparability: views and engagement vary by content type and time window. Document the exact calculation and sample size.
- Privacy: collect only necessary public professional information, avoid sensitive personal data, provide source/retention context, and secure credentials.
- Demo honesty: simulated outreach must be visibly labelled; never represent a simulated send as a real delivery.

Build the smallest end-to-end working path first: campaign → discover → qualify → personalize → simulate → export. A complete, reliable workflow is stronger than many unfinished features.